



**FACULTAD DE
INGENIERÍA**
UNIVERSIDAD DA VINCI
DE GUATEMALA

Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: ingeniería en sistemas



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

“Evaluacion Final”

Eddy Alexander Amperez Carranza

202500054

Curso: Estructura de datos



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

El presente documento contiene cinco fichas técnicas correspondientes a las estructuras arbóreas de datos estudiadas en el curso: Árbol Binario de Búsqueda (BST), Árbol AVL, Árbol B/B+, Trie y Heap. Cada ficha incluye definición, análisis de complejidad algorítmica, características mecánicas, ventajas y limitaciones, y un caso de uso real en la industria.

Link de repositorio: https://github.com/eamperez90/examen_final_arboreas.git



Ficha #01 — Árbol Binario de Búsqueda (BST)

1. Identificación y Definición

Es una estructura de datos jerárquica en la que cada nodo tiene como máximo dos hijos. El hijo izquierdo contiene únicamente valores menores que el nodo padre, y el hijo derecho contiene únicamente valores mayores. Esta propiedad se cumple de forma recursiva en todos los sub-árboles del árbol.

2. Análisis de Complejidad Algorítmica (Big O)

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(n)$	$O(\log n)$
Inserción	$O(n)$	$O(\log n)$
Eliminación	$O(n)$	$O(\log n)$

3. Características Mecánicas

La organización se basa en la propiedad BST: izquierdo < raíz < derecho. Para buscar un valor se compara con el nodo actual y se baja a la izquierda si es menor o a la derecha si es mayor, hasta encontrarlo o llegar a null. La eliminación maneja tres casos: (1) nodo hoja: se elimina directamente; (2) un hijo: el padre apunta al hijo del nodo eliminado; (3) dos hijos: se reemplaza con el sucesor in-order (mínimo del sub-árbol derecho).

4. Ventajas y Limitaciones

Ventaja principal: Búsqueda eficiente en $O(\log n)$ promedio. El recorrido in-order produce los elementos en orden ascendente, lo que facilita operaciones de ordenamiento.

Limitación principal: Sin balanceo automático puede degenerar a $O(n)$ cuando los datos se insertan ya ordenados, comportándose como una lista enlazada.

5. Caso de Uso en la Industria

Software / Sistema: Git · C++ STL (std::map / std::set)

Git utiliza una variante de BST para indexar objetos en su sistema de archivos interno. También es la base de std::map y std::set en la STL de C++.



Ficha #02 — Árbol Balanceado AVL

1. Identificación y Definición

Es un Árbol Binario de Búsqueda con auto-balanceo, propuesto por Adelson-Velsky y Landis en 1962. Para cada nodo, la diferencia de altura entre su sub-árbol izquierdo y derecho (factor de balance) nunca excede $|1|$. Cuando una operación viola esta condición, el árbol realiza rotaciones para restaurar el balance.

2. Análisis de Complejidad Algorítmica (Big O)

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

3. Características Mecánicas

Cada nodo almacena su factor de balance: $FB = \text{altura(derecho)} - \text{altura(izquierdo)}$, el cual debe estar en $\{-1, 0, +1\}$. Cuando se viola el balance se aplican rotaciones: Rotación Simple Derecha ($FB = -2$ e hijo izquierdo con $FB = -1$), Rotación Simple Izquierda (caso espejo), o Rotación Doble cuando los factores tienen signos opuestos. Tras cada operación se recalculan las alturas de abajo hacia arriba.

4. Ventajas y Limitaciones

Ventaja principal: Garantiza $O(\log n)$ en el peor caso para todas las operaciones, a diferencia del BST simple. Es ideal para aplicaciones con muchas búsquedas (lectura intensiva).

Limitación principal: Las rotaciones hacen que la inserción y eliminación sean más lentas que en un BST simple. La implementación es más compleja y requiere almacenar el factor de balance en cada nodo.

5. Caso de Uso en la Industria

Software / Sistema: Linux Kernel · Sistemas de trading HFT

El núcleo de Linux utiliza variantes de AVL para gestionar la memoria virtual (estructuras `mm_struct`). También se usa en sistemas de trading de alta frecuencia para mantener libros de órdenes con acceso garantizado en $O(\log n)$.



Ficha #03 — Árbol B / B+

1. Identificación y Definición

El Árbol B es una estructura auto-balanceada generalizada donde cada nodo puede contener múltiples claves (hasta $m-1$) y múltiples hijos (hasta m), siendo m el orden del árbol. El Árbol B+ es una variante donde todos los datos residen en las hojas (conectadas en lista enlazada) y los nodos internos solo almacenan claves de enrutamiento.

2. Análisis de Complejidad Algorítmica (Big O)

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(\log n)$	$O(\log n)$
Inserción	$O(\log n)$	$O(\log n)$
Eliminación	$O(\log n)$	$O(\log n)$

3. Características Mecánicas

Todas las hojas están al mismo nivel (árbol perfectamente balanceado en altura). Cada nodo interno tiene entre $\lceil m/2 \rceil$ y m hijos. Cuando un nodo se llena ($m-1$ claves), se divide en dos y la clave del medio sube al padre (split). En B+, los nodos internos solo sirven para navegación; los datos reales están en las hojas. Las hojas de B+ están enlazadas, permitiendo recorridos secuenciales eficientes, ideales para consultas de rango en bases de datos (SQL BETWEEN).

4. Ventajas y Limitaciones

Ventaja principal: Minimiza accesos a disco al maximizar datos por nodo (factor de ramificación alto, típicamente entre 100 y 1000). El árbol B+ permite escaneos de rango eficientes gracias a las hojas enlazadas.

Limitación principal: La implementación es compleja, especialmente el manejo de splits y merges. Tiene overhead de espacio en los nodos internos de B+. Es menos eficiente que BST/AVL cuando los datos caben completamente en memoria RAM.

5. Caso de Uso en la Industria

Software / Sistema: MySQL InnoDB · PostgreSQL · NTFS / ext4

Es la estructura fundamental de los motores de bases de datos relacionales. MySQL InnoDB usa B+ trees para todos sus índices primarios y secundarios. Los sistemas de archivos NTFS, ext4 y HFS+ también usan B-trees para sus directorios.



Ficha #04 — Trie (Árbol de Prefijos)

1. Identificación y Definición

Un Trie (del inglés retrieval) es una estructura de árbol especializada para almacenar cadenas de texto. Cada nodo representa un carácter de la cadena. Las claves no se almacenan en los nodos sino que se definen por el camino desde la raíz hasta ese nodo. Todos los descendientes de un nodo comparten el mismo prefijo.

2. Análisis de Complejidad Algorítmica (Big O)

Operación	Peor Caso	Caso Promedio
Búsqueda	$O(m)$	$O(m)$
Inserción	$O(m)$	$O(m)$
Eliminación	$O(m)$	$O(m)$

3. Características Mecánicas

Donde m es la longitud de la clave, independiente de n (número total de claves almacenadas). La raíz es un nodo vacío. Cada nivel representa una posición de carácter. Cada nodo puede tener hasta 26 hijos (alfabeto inglés) o 256 para ASCII completo. Un nodo marcado como `end_of_word` indica que el camino hasta ese nodo forma una palabra válida. Para búsqueda por prefijo, se navega hasta el último carácter del prefijo y todos sus descendientes son palabras con ese prefijo.

4. Ventajas y Limitaciones

Ventaja principal: Búsqueda en $O(m)$ independiente del número de claves almacenadas. El autocompletado y la búsqueda por prefijo son operaciones naturales y eficientes. No requiere función hash ni maneja colisiones.

Limitación principal: Alto consumo de memoria: hasta 26 punteros por nodo aunque la mayoría estén vacíos. Es ineficiente para vocabularios con pocas palabras muy largas y no es adecuado para claves numéricas.

5. Caso de Uso en la Industria

Software / Sistema: Google Search · iOS/Android (teclado predictivo) · Apache Lucene

El autocompletado de Google y los teclados predictivos de iOS y Android utilizan Tries para sugerir palabras en tiempo real. Apache Lucene también los usa internamente para indexación de texto.



Ficha #05 — Heap (Montículo Min/Max)

1. Identificación y Definición

Un Heap es un árbol binario completo que satisface la propiedad de montículo. En un Max-Heap, el valor de cada nodo es mayor o igual al de sus hijos (la raíz es el máximo). En un Min-Heap, el valor de cada nodo es menor o igual al de sus hijos (la raíz es el mínimo). Se implementa eficientemente como un arreglo, donde para el nodo en índice i : hijo izquierdo = $2i+1$, hijo derecho = $2i+2$, padre = $\lfloor (i-1)/2 \rfloor$.

2. Análisis de Complejidad Algorítmica (Big O)

Operación	Peor Caso	Caso Promedio
Inserción	$O(\log n)$	$O(\log n)$
Extraer Min/Max	$O(\log n)$	$O(\log n)$
Ver raíz (peek)	$O(1)$	$O(1)$
Construir heap	$O(n)$	$O(n)$

3. Características Mecánicas

Al insertar, el elemento se coloca al final del arreglo y sube intercambiándose con su padre mientras viole la propiedad heap (sift-up). Al extraer la raíz, el último elemento ocupa su lugar y baja intercambiándose con el hijo de menor o mayor valor (sift-down). La representación como arreglo evita el uso de punteros y es cache-friendly, lo que mejora el rendimiento en la práctica.

4. Ventajas y Limitaciones

Ventaja principal: Acceso al mínimo o máximo en $O(1)$. La implementación como arreglo es eficiente en memoria y cache-friendly. Es la base natural de las colas de prioridad y permite ordenamiento in-place (Heap Sort) en $O(n \log n)$.

Limitación principal: La búsqueda de un elemento arbitrario es $O(n)$ ya que no tiene la propiedad BST. Solo permite acceder eficientemente al extremo (mínimo o máximo), no a elementos intermedios.

5. Caso de Uso en la Industria

Software / Sistema: Google Maps (Dijkstra) · Linux Kernel (scheduler) · Compresión ZIP/GZIP

El algoritmo de Dijkstra para rutas más cortas utiliza un Min-Heap como cola de prioridad. El scheduler CFS del kernel de Linux y la compresión Huffman (usada en ZIP, GZIP, JPEG) también utilizan heaps.



FACULTAD DE INGENIERÍA

UNIVERSIDAD DA VINCI
DE GUATEMALA